Hyacinth: Electrical Tail Completion for MoE Inference over Optical Interconnects

Heng Xu[1], Ying Zhou[1], Zhonghua Peng[1], Jialong Sun[1], Yang Jiao[1], Jialong Li[1,✉]
[1] Faculty of Computer Science and Artificial Intelligence, Shenzhen University of Advanced Technology
✉lijialong@suat-sz.edu.cn

***Abstract*—Mixture-of-Experts (MoE) inference stresses data-center interconnects with sparse and highly skewed variable-size all-to-all collective (AlltoAllv) traffic, making collective completion time a key network-level metric. Optical circuit switching (OCS) provides high bandwidth and low power, but pure-optical scheduling struggles with residual flows that miss direct optical links and require costly multi-hop detours. Existing optical-electrical designs add flexibility, yet their static traffic partitioning is poorly suited to state-dependent residual-flow completion. We present Hyacinth, an online residual-flow completion scheduler for optical inference fabrics with a small electrical completion layer. Hyacinth does not propose a new topology. Instead, it jointly evaluates optical paths and electrical completion paths under a unified completion-time framework. OCS remains the primary carrier for well-matched flows, while electrical paths are exposed only as bounded candidates for residual flows. Hyacinth then selects paths using link-state scores that combine reserved load and predicted completion time, avoiding deep optical search while adapting electrical use to runtime contention. With Hyacinth-dynamic as the default configuration, Hyacinth reduces flow completion time by up to 32.6% for large flows and 9.0% for small flows over Pure Optics baselines, and reduces collective completion time by up to 75.8% over static-threshold optical-electrical approaches.**

***Index Terms*—Optical circuit switching, electrical completion layer, AlltoAllv communication, Mixture-of-Experts inference.**

I. Introduction

LARGE language model inference, especially Mixture-of-Experts (MoE) inference, is moving data-center interconnects toward a new operating point. In MoE models [28], [52], [57], [58], [6], expert parallelism routes tokens to a small subset of experts, producing variable-size all-to-all collective (AlltoAllv) traffic that is sparse, skewed, and load imbalanced. Meanwhile, online MoE inference [60], [38], [41], [50], [32] repeatedly invokes synchronized AlltoAllv communication rounds, where the next computation phase cannot proceed until the slowest transfer in the round completes. This makes collective completion time, especially its tail, a first-order network bottleneck. Because similar expert-routing patterns repeat across decode steps, a small number of slow residual flows can delay every communication round and repeatedly expose the interconnect to tail completion pressure.

Optical Circuit Switching (OCS) is an attractive primary interconnect for this setting. Compared with Electronic Packet Switching (EPS), which provides flexible connectivity but incurs higher packet-processing overhead and power consumption, OCS can deliver high bandwidth with low per-bit cost and power, while adding only small deterministic latency [40], [23], [3], [36]. These properties make an optical inference fabric a promising substrate for carrying the bulk of expert-parallel traffic. However, the benefits of OCS are most direct when flows can use short optical paths, especially direct optical links between communicating racks [7], [31].

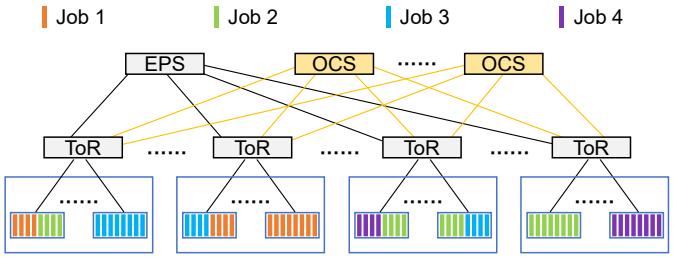

Fig. 1: Fragmented scenarios during MoE inference.

This condition is difficult to maintain under load-imbalanced MoE inference traffic. Each Top-of-Rack (ToR) switch can maintain only a limited number of optical direct links, so only a subset of source-destination pairs can be served by high-quality optical paths in each scheduling round [8], [34], [35], [25]. Flows that miss these links become residual tail flows. Pure-optical scheduling must then search for multi-hop optical detours to serve them. As the allowed hop count increases, the search space grows rapidly, while the remaining flows are often small, fragmented, and highly sensitive to waiting on busy optical links. The result is a mismatch between the efficiency of OCS for well-matched flows and the tail latency caused by hard-to-serve residual flows. Fig. 1 illustrates the multi-tenant inference, where fragmented jobs run on an optical fabric supplemented by a small electrical completion layer.

A small amount of electrical connectivity offers a different way to handle this tail. Its value is not high aggregate bandwidth, but full reachability through short completion edges. Existing optical-electrical designs, however, often partition traffic between the optical and electrical planes before path selection, for example by assigning large or scheduled demands to circuit paths and leaving the remaining traffic to packet paths [15], [54], [29], [9]. Such pre-partitioning is premature for MoE inference. A flow's best path depends not only on its size, but also on the current availability of optical links and the possibility of using a short electrical completion path. This paper focuses on a narrower scheduling problem: *given a degree-limited optical inference fabric and a small electrical completion layer, how can an online scheduler complete residual MoE flows that miss short optical paths without resorting to expensive deep optical search?*

To bridge this gap, we present Hyacinth, an online residual-flow completion scheduler for MoE inference over optical interconnects. Hyacinth abstracts the OCS subgraph and the electrical completion layer into a unified path graph, but treats electrical connectivity as a bounded completion resource rather than a separate traffic plane. The key insight is that adding a small number of electrical completion edges to the same completion-time analysis reduces the scheduler's search burden for residual flows. Instead of expanding ever-deeper optical detours, Hyacinth compares short optical paths and electrical completion paths with the same link-state metrics, including reserved load and predicted completion time. By using the electrical layer only when it improves the path score, Hyacinth preserves efficient direct OCS paths for well-matched traffic while keeping the online candidate space small.

Central to Hyacinth's design is the combination of a theoretical model and an engineering realization. We model the OCS subgraph as a K-regular graph among ToRs and the electrical layer as N fully connected EPS nodes attached to all ToRs. This model yields a path-reservation formulation for Pure Optics and optical scheduling with electrical completion, which share the same objective and reservation constraints but differ in path space. Guided by this structure, Hyacinth avoids solving a global mixed-integer reservation model online. Instead, it builds a lightweight scheduler over the unified path space. The scheduler enumerates short optical paths and electrical completion paths, bounds the retained candidate set, predicts completion time from link availability and bottleneck rates, and updates link states through per-flow scheduling with strict link reservation.

We evaluate Hyacinth using packet-level simulation across two topology scales and four traffic scenarios, including fragmentation, concurrency conflict, heterogeneous model composition, and workload distribution shifts. In the main comparison, Hyacinth-dynamic reduces average collective completion time (CCT) by about 14% to 28% at 2560 GPUs and by 7% to 21% at 5120 GPUs compared with Pure Optics baselines. Against Helios, an optical-electrical baseline with a static threshold, the reduction reaches up to 75.8% under high concurrency. Hyacinth-pruned further improves CCT by 13.3% at 2560 GPUs, showing its value for sparse OCS fabrics. These results show that a small electrical completion layer is enough to improve completion time across diverse traffic conditions and scales when it is scheduled jointly with optical paths.

This paper makes the following contributions.

- We identify a core tension in optical interconnects for MoE inference, as OCS efficiently serves flows that match direct optical links, while the round tail is often dominated by residual flows that miss these links and require deep optical path search with rapidly diminishing returns.
- We propose Hyacinth, an online residual-flow completion scheduler for optical inference fabrics with a small electrical completion layer. Hyacinth places optical paths and electrical completion paths in the same completion-time framework, uses the electrical layer only when it improves the current path score of residual flows, and controls online solving cost through bounded candidate construction, completion-time prediction, and strict link reservation.

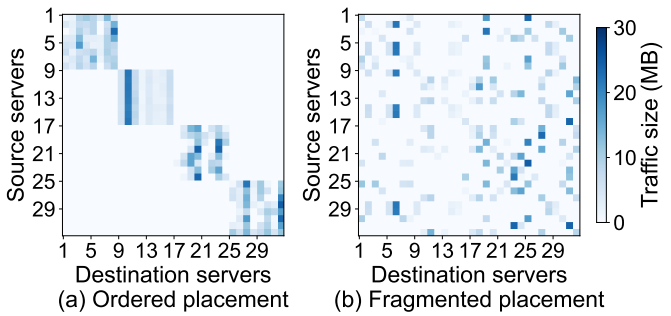

Fig. 2: (a) Idealized versus (b) fragmented AlltoAllv demand. Ordered placement is an idealized reference, while practical multi-tenant scheduling often forces models onto non-contiguous servers, scattering sparse and skewed AlltoAllv demand into residual flows beyond limited OCS direct links.

- We implement Hyacinth in a packet-level flow simulation framework and evaluate it across two topology scales and four traffic scenarios. Results show that the default Hyacinth-dynamic reduces average CCT compared with Pure Optics and optical-electrical baselines with static thresholds, while Hyacinth-pruned provides an enhanced mode for sparse OCS fabrics.

II. Background and Motivation

In this section, we analyze how load-imbalanced AlltoAllv traffic stresses OCS interconnects (§II-A), discuss why a small electrical layer should serve tail completion rather than a separate traffic plane (§II-B), and outline how these observations motivate Hyacinth's design (§II-C).

A. Residual Tail Flows in MoE AlltoAllv

We begin by examining why MoE inference creates tail-critical traffic for optical interconnects. Distributed LLM inference uses multiple parallel strategies with different communication patterns. Data parallelism (DP) [26], [44] replicates model instances across request batches and, unlike training, does not require gradient synchronization during inference. Pipeline parallelism (PP) [37], [20] transfers activations between stages, and tensor parallelism (TP) [48] performs fine-grained collectives within each layer. In contrast, expert parallelism (EP) [57], [28], [14], which is central to MoE, routes tokens to experts through AlltoAllv and produces sparse, skewed inter-node transfers.

Among inference-time communication patterns, EP and TP are the two major sources of traffic volume, but they stress the network in different ways. TP communication is typically confined within tightly placed GPU groups and is often served within a server or a rack, whereas EP routes tokens to distributed experts and produces sparse, skewed AlltoAllv transfers across ToRs. Fig. 2 illustrates this effect. Ordered placement is an idealized reference that exposes the sparse and skewed structure of model-local AlltoAllv demand. In practical multi-tenant settings, however, models often occupy non-contiguous servers, scattering the same demand into fragmented residual flows. Since similar expert-routing patterns

repeat across decode steps, any per-round straggler from load-imbalanced EP traffic can cascade into sustained tail latency.

In this tail-critical regime, OCS is most effective when flows match a small number of high-value direct links. Each ToR can maintain only a limited number of optical direct links, so only a subset of source-destination pairs can be mapped to high-quality OCS edges in a scheduling round. These matched flows enjoy short, high-rate paths. The remaining pairs form the long tail of the AlltoAllv traffic matrix and must rely on multi-hop optical paths.

This creates the central tension for Pure Optics. From a reachability perspective, deeper optical paths can serve more residual flows. From an online scheduling perspective, deeper search quickly becomes expensive. In a K-regular graph, depth-h path expansion can examine $O(K^h)$ candidates. At the same time, the additional flows reached by deeper search are increasingly fragmented and more likely to wait on late-released links or shared bottlenecks. The key issue is therefore not whether a path exists, but whether continued optical search is worthwhile for a diminishing fraction of tail flows.

For MoE inference, this tension is amplified by the traffic pattern itself. Expert selection is input-dependent and non-uniform, producing a heavy-tailed flow-size distribution in which a few hot expert pairs carry most data while many pairs carry little. During autoregressive decoding, similar token-to-expert patterns repeat over many decode steps, so a single slow communication round can repeatedly affect inference latency. The interconnect challenge is therefore not steady-state throughput balancing, but minimizing the per-round completion tail under sparse, skewed, and imbalanced AlltoAllv traffic.

B. Electrical Layer as Tail Completion

1) Avoid traffic pre-partitioning. After pure-optical search reaches diminishing returns, the missing capability is not simply more deep optical paths, but a way to complete residual flows with bounded candidate search. A small electrical completion layer provides this capability through full reachability. Conventional optical-electrical designs often partition traffic between planes before path selection, assigning large or scheduled demands to circuit paths and the remaining traffic to packet paths [15], [54], [29]. This is premature for MoE inference AlltoAllv because a flow's best path depends on current OCS and electrical link availability, not only on size or class. Pre-partitioning also removes mixed completion paths that combine an OCS segment with an electrical completion edge, forcing a residual flow into one plane even when a shorter and earlier-completing path exists.

2) Make electrical use state dependent. A static threshold for electrical use is also difficult to choose. The right amount of electrical participation depends on the optical degree, current link occupancy, and the traffic mix in the current AlltoAllv round. A threshold that is useful in a sparse OCS graph can overload the electrical uplink in a denser graph, while a conservative threshold can leave residual flows waiting for deep optical detours. Thus the key decision is not only which electrical completion candidates to expose, but also when they should be used. This motivates treating electrical use as an online completion-time comparison rather than as a static flow-size split.

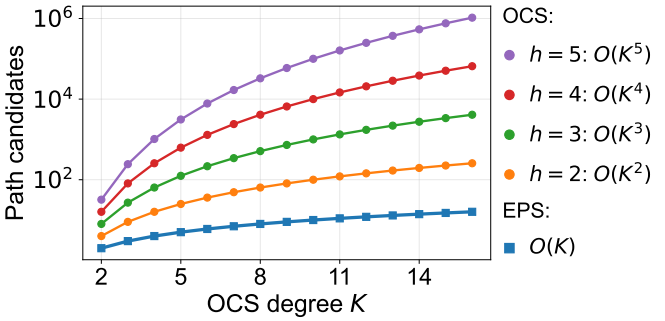

Fig. 3: Pure Optics search can grow as $O(K^h)$ with hop-count cutoff h, while electrical completion provides $O(K)$ local candidates for residual flows.

3) Contract deep optical search. The electrical layer is most useful as a bounded completion resource, not as a preset traffic plane. For a flow whose source and destination have minimum directed OCS distance h in a K-regular optical graph, Pure Optics may examine $O(K^h)$ candidates. With one traversal through a fully connected EPS node, direct electrical completion plus OCS-then-electrical and electrical-then-OCS options expose at most $2K + 1 = O(K)$ local candidates, as shown in Fig. 3. The benefit is algorithmic. Short electrical completion options can replace the deepest optical detours, making search depth a small constant rather than a function of graph diameter.

4) Optimize one completion objective. The electrical layer should therefore be treated as a tail-completion layer inside an optical scheduling problem, not as an independent plane for a predefined traffic class. OCS direct links and electrical completion options should be placed in one unified path graph, so that both serve the same online completion-time objective.

C. Design Implications

These observations directly shape the design of Hyacinth. First, the electrical layer should be introduced as a tail-completion option within optical scheduling, rather than assigned a preset traffic class in advance. Second, the scheduler should compare concrete optical paths and electrical completion paths under current link states, instead of relying on static rules such as flow size or path type. Third, the search scale must be explicitly controlled. Finally, electrical use should adapt to link occupancy so that the scheduler avoids both underusing the completion layer and overloading its uplinks. The goal is not to find ever-deeper optical paths, but to use short electrical completion paths to serve residual tail flows when they lead to earlier completion. Following this reasoning, the next section formalizes the problem with a unified graph model, and the design section realizes it as a lightweight online scheduler over a bounded candidate space.

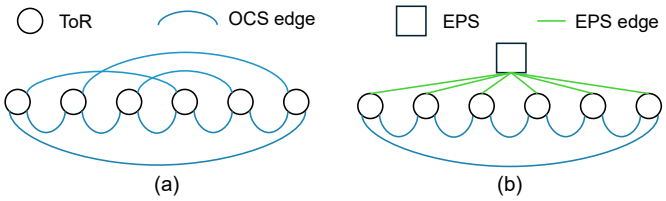

Fig. 4: Unified graph for (a) OCS-only scheduling and (b) optical scheduling with electrical completion.

III. Problem Formulation

This section formalizes residual-flow scheduling over an optical fabric with a small electrical completion layer. We define the path graph (§III-A), a path-reservation formulation (§III-B), and four formal results that support online scheduling. They cover electrical completion coverage, search contraction, retained-set scoring cost, and state-dependent electrical participation (§III-C–§III-D).

A. System Model

In Fig. 4, we model the core network as an optical fabric with a small electrical completion layer. The first layer is an OCS subgraph among ToR switches. The second layer is a small number of EPS nodes that supplement the OCS fabric with packet-switched reachability. This structure is motivated by deployments where OCS provides high-bandwidth paths between a limited number of ToR pairs, while the electrical layer provides short completion paths when the optical graph does not offer a short available route.

OCS subgraph. The OCS subgraph connects M ToRs through circuit-switched optical links. We model each usable direction as a directed edge. Logically, each ToR has K outgoing and K incoming optical edges, forming a K-regular directed graph. In practice, this regular structure is realized through pre-configured Micro-Electro-Mechanical Systems (MEMS) [46], [31] OCS fabrics. We denote the set of ToRs as $R = \{1, 2, \ldots, M\}$ and the OCS subgraph as $G_O = (R, L_O)$, where each directed edge $\ell \in L_O$ carries traffic at rate c^O.

Electrical completion layer. The system deploys N EPS nodes, denoted $S = \{e_1, \ldots, e_N\}$, each bidirectionally connected to every ToR. These EPS-ToR edges operate at rate c^E. We allow heterogeneous link rates with $c^O \geq c^E > 0$, while the evaluation also considers the equal-rate case to isolate the effect of path selection. We model contention on EPS-ToR links and assume each EPS node is nonblocking at the modeled port rates. In our evaluated configuration, we use $N = 1$ and $c^E = c^O$, so each ToR has one electrical completion uplink whose bandwidth matches one OCS port. This reflects the supplementary role of the electrical layer. Each ToR thus has K OCS uplinks and N electrical completion uplinks for a total core degree of $K + N$.

Unified graph. The two layers combine into a single directed graph

$$G_H = (V, L_H), \qquad V = R \cup S,\ L_H = L_O \cup L_E, \tag{1}$$

where L_E denotes directed edges between EPS nodes and ToRs. This unified path graph places OCS links and electrical completion edges in a shared topology. Links differ in rate, which the graph explicitly encodes, but there is no a priori designation of which traffic classes belong to which plane. The scheduler observes the full graph and makes path decisions per flow.

AlltoAllv collective model. An AlltoAllv collective round J arrives at time A_J and consists of a set of flows F_J. Each flow $f \in F_J$ is a four-tuple (s_f, d_f, b_f, a_f), specifying source ToR, destination ToR, volume in bits, and arrival time. Flows in the same synchronized AlltoAllv round are released together. The formulation below analyzes this round-level case with $a_f = A_J$. The online scheduler keeps a_f explicitly so staggered and overlapping collective rounds can be handled by the same rule. The per-ToR-pair demand aggregates to an $M \times M$ matrix

$$D^J = (d^J_{uv})_{M\times M}, \qquad d^J_{uv} = \sum_{f\in F_J : s_f = u, d_f = v} b_f. \tag{2}$$

The objective is to minimize collective completion time

$$T_J = \max_{f\in F_J} C_f, \tag{3}$$

where C_f denotes the completion time of flow f. Since inference communication proceeds in synchronized AlltoAllv rounds, T_J captures the network-side tail completion time of each collective round.

Link state. The scheduler maintains per-link state. For each link $\ell \in L_H$, let τ_ℓ record the earliest time at which ℓ becomes available, accounting for all previously scheduled flows. When collective round J arrives at time A_J, the residual busy time on link ℓ is

$$\bar{\tau}_\ell = \max(0, \tau_\ell - A_J), \qquad \forall \ell \in L_H. \tag{4}$$

A link with $\bar{\tau}_\ell = 0$ is immediately free. A positive value means the link is occupied by prior collective rounds and will not accept new traffic until $A_J + \bar{\tau}_\ell$. These per-link busy states capture the dynamic contention that accumulates as collective rounds arrive and depart.

B. Path Reservation Formulation

We next state the offline path-reservation problem that Hyacinth approximates online. The formulation is intentionally integral. Each flow selects one path, reserves every link on that path, and two flows that share a link must be ordered on that link. This captures the route-level decision made by the scheduler, while packet-level queueing and congestion control are evaluated later in simulation.

Let $\mathcal{F}$ be the set of flows in a scheduling window and let $\mathcal{Q}$ be the set of collective rounds represented in that window. The mapping $\gamma(f) \in \mathcal{Q}$ gives the round that contains flow f. For topology type $X \in \{O, H\}$, O denotes Pure Optics and H denotes the optical fabric with electrical completion. Let L^X be the link set and P^X_f be the admissible path set for flow f. For a path $p \in P^X_f$, define

$$r(p) = \min_{\ell\in p} c_\ell, \qquad d_{fp} = \frac{b_f}{r(p)}. \tag{5}$$

Let $\delta_{\ell p} = 1$ if link ℓ is on path p, and $\delta_{\ell p} = 0$ otherwise. Let θ_ℓ^X be the initial availability time of link ℓ at the start of the window, inherited from earlier reservations. If there are no earlier reservations, $\theta_\ell^X = 0$.

The decision variables are as follows. The binary variable $x_{f,p}^X$ selects path p for flow f. The binary variable $y_{f,\ell}^X$ records whether flow f uses link ℓ. The binary variable $z_{\ell ff'}^X$ orders two flows on a shared link. The continuous variables u_f^X and C_f^X are the start and completion times of flow f, and C_q^X is the completion time of collective round q. Let $\mathcal{M}$ be a sufficiently large constant. The reservation problem is

$$\min \quad \sum_{q\in\mathcal{Q}} C_q^X \tag{6a}$$

$$\text{s.t.} \quad \sum_{p\in P_f^X} x_{f,p}^X = 1, \qquad \forall f \in \mathcal{F}, \tag{6b}$$

$$y_{f,\ell}^X = \sum_{p\in P_f^X} \delta_{\ell p} x_{f,p}^X, \qquad \forall f \in \mathcal{F},\ \ell \in L^X, \tag{6c}$$

$$u_f^X \geq a_f, \qquad \forall f \in \mathcal{F}, \tag{6d}$$

$$u_f^X \geq \theta_\ell^X - \mathcal{M}(1 - y_{f,\ell}^X), \qquad \forall f \in \mathcal{F}, \ell \in L^X, \tag{6e}$$

$$C_f^X \geq u_f^X + \sum_{p\in P_f^X} d_{fp} x_{f,p}^X, \qquad \forall f \in \mathcal{F}, \tag{6f}$$

$$u_{f'}^X \geq C_f^X - \mathcal{M}(3 - y_{f,\ell}^X - y_{f',\ell}^X - z_{\ell ff'}^X), \qquad \forall f < f', \ell \in L^X, \tag{6g}$$

$$u_f^X \geq C_{f'}^X - \mathcal{M}(2 - y_{f,\ell}^X - y_{f',\ell}^X + z_{\ell ff'}^X), \qquad \forall f < f', \ell \in L^X, \tag{6h}$$

$$C_q^X \geq C_f^X, \qquad \forall f \in \mathcal{F},\ \gamma(f) = q, \tag{6i}$$

All path-selection, link-use, and ordering variables are binary. Start times and completion times are nonnegative continuous variables.

Constraint (6b) chooses one path per flow. Constraint (6c) derives link usage from the chosen path. Constraints (6d)–(6e) enforce flow arrival and inherited link availability. Constraint (6f) defines flow completion time. Constraints (6g) and (6h) ensure that two flows sharing a link are serialized on that link. Constraint (6i) defines collective completion time. When the window contains a single collective round, the objective reduces to minimizing its CCT.

This formulation gives the right comparison point for Hyacinth. Pure Optics and electrical completion scheduling use the same objective and reservation constraints. They differ in the link set L^X, path set P_f^X, and link rates c_ℓ. Importantly, Hyacinth does not rely on a feasible-region dominance claim. If electrical links were added without removing any OCS links, the completion graph would contain every Pure Optics schedule. Our evaluation uses the stricter degree-constrained replacement model, where one electrical uplink replaces one OCS uplink. In this setting, path-space containment need not hold. The structural question is instead whether the electrical completion layer gives useful local candidates without requiring deep optical search.

C. Structural Analysis

The reservation formulation separates the objective from path-space construction. We now analyze the path-space property that remains valid under the degree-constrained replacement model used in evaluation. Under a per-ToR core degree budget D, Pure Optics spends all D ports on OCS links. Hyacinth uses $K = D - 1$ OCS ports and one electrical completion uplink per ToR. This sacrifices one unit of optical degree, but creates a bounded local completion set for every ordered ToR pair.

Theorem 1 (Degree-Constrained Electrical Completion Coverage)**.** *Consider a degree-constrained optical fabric with per-ToR core degree $D = K + 1$. Each ToR has K outgoing and K incoming OCS edges, and one nonblocking EPS node e is bidirectionally connected to every ToR as the electrical completion layer. For any ordered ToR pair (s, d) with $s \neq d$, define*

$$\begin{aligned}\mathcal{E}_{s,d} = &\{s \to e \to d\} \\ &\cup \{s \to u \to e \to d : (s,u) \in L_O,\ u \neq d\} \\ &\cup \{s \to e \to v \to d : (v,d) \in L_O,\ v \neq s\}.\end{aligned} \tag{7}$$

Every path in $\mathcal{E}_{s,d}$ is feasible in the completion graph. Moreover, $|\mathcal{E}_{s,d}| \leq 2K + 1$, independent of the directed OCS distance from s to d.

Proof. The direct path $s \to e \to d$ exists because every ToR has a bidirectional link to e. For each outgoing OCS neighbor u of s, the edge $s \to u$ exists by definition, and $u \to e \to d$ exists through the EPS node. Thus $s \to u \to e \to d$ is feasible whenever $u \neq d$. Similarly, for each incoming OCS neighbor v of d, the path $s \to e \to v \to d$ is feasible whenever $v \neq s$. Since the OCS subgraph is K-regular, s has at most K outgoing OCS neighbors and d has at most K incoming OCS neighbors. Removing invalid or duplicate candidates can only reduce the count, so $|\mathcal{E}_{s,d}| \leq 1 + K + K = 2K + 1$. The construction uses only the local OCS neighborhoods of s and d, so the bound does not depend on their OCS distance. ■

Corollary 1 (Degree-Constrained Search Contraction)**.** *In the corresponding Pure Optics fabric with the same per-ToR degree budget D, suppose a residual ToR pair has shortest directed optical distance h. A naive Pure Optics depth-h expansion can examine $\sum_{j=1}^{h} D^j = O(D^h)$ directed partial paths before retaining paths that terminate at the destination. In the degree-constrained optical fabric with electrical completion from Theorem 1, where $K = D - 1$, the electrical completion candidates for the same ordered pair can be enumerated in $O(K) = O(D)$ time and have size at most $2K + 1$, independent of h.*

Proof. Each Pure Optics expansion step has at most D outgoing choices. For $D > 1$, the number of directed partial paths explored up to depth h is bounded by

$$N_f(h) \leq \sum_{j=1}^{h} D^j = \frac{D^{h+1} - D}{D - 1}, \tag{8}$$

which is $O(D^h)$. The $D = 1$ case is linear. Theorem 1 constructs the electrical completion set from the outgoing OCS neighbors of s and the incoming OCS neighbors of d. This requires scanning at most $2K$ local OCS neighbors plus the direct electrical option, giving $O(K)$ enumeration time and at most $2K + 1$ candidates. Since $K = D - 1$, the electrical completion enumeration cost is also $O(D)$. ■

Theorem 1 and Corollary 1 are deliberately limited. They are coverage and search-cost statements, not performance guarantees. An electrical uplink can become busy, and a removed OCS link can be useful for some flows. The scheduler must therefore compare optical and electrical completion candidates under current link state.

These two results directly motivate the path-space design in Section IV-A. The candidate path set should include the three electrical completion structures in Eq. (7) alongside controlled OCS candidates. In the pruned construction, this yields the six path families used by Hyacinth. The scheduler should then compare concrete paths using common link-state metrics rather than pre-assigning traffic to a plane. The $O(K)$ completion bound keeps electrical search independent of graph diameter.

D. Local Scheduling Metric

The path-reservation formulation gives a common completion-time objective, but solving it online is impractical because it contains binary path decisions and pairwise link-ordering variables. We therefore use local scheduling metrics that retain the two quantities most visible in the formulation, namely link availability and bottleneck rate.

For any candidate path p, the earliest time at which all links on p become available is

$$\rho(p) = \max_{\ell \in L_H : \delta_{\ell p} = 1} \tau_\ell. \tag{9}$$

The bottleneck transmission rate along p is

$$r(p) = \min_{\ell \in L_H : \delta_{\ell p} = 1} c_\ell. \tag{10}$$

Both quantities are determined by the most constrained link on the path. We use a fluid path model in which a flow occupies all links on p after they become available and its throughput is limited by the bottleneck link. Under this model, if flow f is assigned to path p, its predicted local completion time is

$$\widehat{C}(f,p) = \max\{a_f, \rho(p)\} + \frac{b_f}{r(p)}. \tag{11}$$

This prediction is local in two senses. It considers only links on p, not the global coupling across flows in the reservation formulation. It also ignores future arrivals. Nevertheless, it preserves the same basic tradeoff, waiting for busy links versus transmitting at the path bottleneck rate. When the scheduler selects a path and immediately updates link states via $\tau_\ell \leftarrow \widehat{C}(f,p^\star)$ for each link on the chosen path, subsequent flows perceive an internally consistent link state. This gives a lightweight online rule aligned with the formulation, without claiming a formal approximation ratio.

For the dynamic mode, Hyacinth also tracks accumulated reserved service q_ℓ on each link and scores the maximum post-assignment load

$$w(p) = \max_{\ell \in p} \left(q_\ell + \frac{b_f}{r(p)} \right). \tag{12}$$

The online scheduler evaluates this metric over a retained candidate set rather than the full path set. Let $\tilde{P}_f$ be the retained set for flow f after keeping at most K_{type} candidates from each of the six path families used by Hyacinth, namely one-hop, two-hop, and three-hop Pure Optics paths plus the three electrical completion families in Eq. (7).

Lemma 1 (Bounded Retained-Set Scoring)**.** *If every path family retains at most K_{type} candidates and each retained path has at most three network hops, then evaluating the retained set under $\rho(p)$, $r(p)$, $\widehat{C}(f,p)$, and $w(p)$ costs $O(K_{type})$ time per flow.*

Proof. There are six path families, so $|\tilde{P}_f| \le 6K_{\text{type}}$. Each retained path has constant length, at most three network hops in the path families used by Hyacinth. Computing $\rho(p)$ and $r(p)$ scans the links on p, which is $O(1)$. Computing $\widehat{C}(f,p)$ and the dynamic load score then uses these path-local quantities and is also $O(1)$. Evaluating all retained candidates therefore costs $O(6K_{\text{type}}) = O(K_{\text{type}})$. ■

Lemma 1 only bounds scoring after candidate retention. Candidate generation can still inspect more raw OCS candidates before capping, especially for three-hop Pure Optics paths. This distinction matches the design in Section IV-B.

Theorem 2 (State-Dependent Electrical Threshold)**.** *Let o be the best retained Pure Optics candidate and e be the best retained electrical completion candidate under Eq. (11). Define $T_o = \max\{a_f, \rho(o)\}$ and $T_e = \max\{a_f, \rho(e)\}$. Under completion-time scoring, the electrical candidate is preferred exactly when*

$$T_e + \frac{b_f}{r(e)} < T_o + \frac{b_f}{r(o)}. \tag{13}$$

If $r(o) > r(e)$, this is equivalent to

$$b_f < \frac{T_o - T_e}{1/r(e) - 1/r(o)}. \tag{14}$$

If the rates are equal, the decision reduces to $T_e < T_o$.

Proof. The electrical candidate has lower predicted completion time than the Pure Optics candidate exactly when Eq. (13) holds. Rearranging gives

$$b_f \left(\frac{1}{r(e)} - \frac{1}{r(o)} \right) < T_o - T_e. \tag{15}$$

When $r(o) > r(e)$, the coefficient of b_f is positive, which gives Eq. (14). When $r(o) = r(e)$, the size-dependent terms cancel, and the decision depends only on whether $T_e < T_o$. ■

Theorem 2 explains why electrical participation is a dynamic threshold rather than a static flow-size split. The threshold depends on current link availability as well as path rates. When optical links are occupied, T_o increases and electrical

completion becomes more attractive. When the electrical uplink is occupied, T_e increases and the scheduler naturally shifts traffic back to optical paths.

Critically, $\rho(p)$, $r(p)$, $\widehat{C}(f,p)$, and $w(p)$ are defined uniformly over Pure Optics and electrical completion paths. There is no need to pre-classify flows or preselect a plane. The pruned scheduler directly minimizes the completion-time metric, while the dynamic scheduler adds the min-max reserved-load score before using $\widehat{C}(f,p)$ as a secondary term. In both cases, electrical participation is determined by the current path score rather than a preset traffic class.

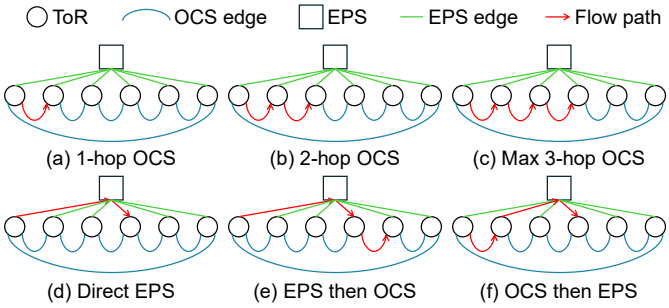

Fig. 5: Six path types for Hyacinth.

IV. Design and Implementation

In this section, we describe how Hyacinth realizes the theoretical framework of Section III-C as a lightweight online scheduler. We first outline the two tensions that drive the design (§IV-A), present the path space construction that instantiates Theorem 1 and Corollary 1 (§IV-B), describe unified selection rules derived from the local metrics and threshold result in Section III-D (§IV-C), and give two complete scheduling algorithms, a dynamic greedy scheduler and a pruned search variant (§IV-D).

A. Design Overview

Hyacinth schedules AlltoAllv flows online over a preconfigured optical topology with a small electrical completion layer. It does not address OCS topology generation or dynamic circuit reconfiguration. The input is a stream of flows from arriving inference collective rounds. The output is a path assignment and link reservation for each flow, made immediately upon arrival.

Translating Section III-C into an online procedure requires resolving two tensions. First, electrical completion gives $O(K)$ local alternatives, but the scheduler still needs bounded OCS search and a controlled way to combine the two candidate sets. Exhaustive OCS expansion and unconstrained electrical combinations would still be too costly. Second, path value is state dependent. An OCS path may be blocked by prior reservations, while an electrical completion path may contend on its uplink. The right choice is determined by predicted completion time and link occupancy, not by path class. This is the design form of the threshold problem in Section II-B.

Hyacinth maps the four observations in Section II-B to four coupled design choices.

1) Unified path selection. This answers the pre-partitioning observation. Hyacinth evaluates OCS and electrical completion paths with the same link-state metrics, so selection follows the current score rather than a preset plane assignment.

2) Dynamic scoring. This answers the threshold observation. Electrical participation moves with reserved load and predicted completion time, rather than a static flow-size threshold. This follows the state-dependent comparison in Theorem 2, with the dynamic mode adding reserved-load pressure to protect shared uplinks.

3) Bounded path construction. This answers the search-contraction observation. Hyacinth fixes a small set of OCS and electrical completion path families, giving each flow a bounded candidate space that instantiates Theorem 1 and Corollary 1. Per-family capping then gives the retained-set bound used by Lemma 1.

4) Immediate link reservation. This keeps both path types inside one completion objective. After each selection, Hyacinth updates link availability so later flows observe both electrical contention and OCS occupancy. Together, these choices keep electrical completion inside optical scheduling while limiting deep optical search.

Hyacinth exposes these choices through two modes. Hyacinth-dynamic is the default production-oriented mode, using capped pools and min-max load scoring for low online overhead. Hyacinth-pruned is an enhanced mode for sparse or low-degree OCS, spending extra search when route quality can offset the cost.

Together, these choices realize a **dynamic threshold**. Flows are not pre-assigned to OCS or the electrical layer by flow size. Electrical participation is decided per flow and per path by the current score. An electrical completion path is chosen only when it outperforms the available Pure Optics alternatives under current link state. The threshold therefore moves with link occupancy, unlike static-threshold designs such as Helios [15].

B. Path Space Construction

Theorem 1 identifies three electrical completion structures that collectively provide up to $2K+1$ local candidates between any source-destination pair via the EPS node. In addition, Hyacinth introduces three classes of bounded-depth Pure Optics paths, resulting in a total of six path types in Fig. 5.

$$\tilde{P}_f \subseteq \left\{ \begin{matrix} s_f \to d_f,\ s_f \to k \to d_f,\ s_f \to k_1 \to k_2 \to d_f, \\ s_f \to e \to d_f,\ s_f \to k \to e \to d_f,\ s_f \to e \to k \to d_f \end{matrix} \right\}, \tag{16}$$

where $k, k_1, k_2 \in R$, $e \in S$, and each listed OCS hop is included only when the corresponding edge exists in G_O. The first row contains one-hop, two-hop, and three-hop Pure Optics paths. The second row contains direct electrical completion, OCS-then-electrical, and electrical-then-OCS completion structures. Every path in the second row traverses the EPS node exactly once and uses at most one OCS hop. These six families are deliberately bounded rather than exhaustive. They cover the short OCS paths that Hyacinth searches online and ensure at least one electrical completion route for every ToR pair. Deeper OCS paths may exist, but including them

increases search cost and often exposes the flow to more shared bottlenecks.

Per-class capping. Path families alone are insufficient because different families have different cardinalities. In a K-regular graph, three-hop Pure Optics can require inspecting $O(K^2)$ candidates while direct electrical completion generates exactly one. Pooling all candidates naively biases selection toward families with more candidates. Hyacinth therefore applies uniform capping. Within each family, candidates are ranked by a composite score favoring earlier link availability, higher bottleneck rate, and shorter length, and only the top K_{type} are retained, with the default set to 8. The retained candidate set per flow is bounded by $6 \cdot K_{\text{type}}$, which is constant for a given cap and independent of graph diameter. Lemma 1 bounds the scoring cost on this retained set. Candidate generation can still inspect $O(K^2)$ three-hop OCS paths before capping.

Algorithm 1: Hyacinth-dynamic

Input : Flow $f = (s_f, d_f, b_f, a_f)$, graph G_H, link states $\{\tau_\ell, q_\ell\}$, cap K_{type}
Output: Assigned path $p_f^\star$, updated link states

Retrieve a capped pool of up to K_{type} shortest OCS paths in G_O from s_f to d_f;
Retrieve up to K_{type} electrical completion candidates from the three completion families;
foreach *candidate p from both pools* **do**
  $\rho(p) \leftarrow \max_{\ell \in p} \tau_\ell$;
  $r(p) \leftarrow \min_{\ell \in p} c_\ell$;
  $\widehat{C}(f,p) \leftarrow \max\{a_f, \rho(p)\} + b_f / r(p)$;
  $w(p) \leftarrow \max_{\ell \in p}(q_\ell + b_f / r(p))$;
Select $p_f^\star$ by $(w(p), \widehat{C}(f,p), \rho(p), -r(p), |p|)$;
foreach *link $\ell \in p_f^\star$* **do**
  $\tau_\ell \leftarrow \widehat{C}(f, p_f^\star)$;
  $q_\ell \mathrel{+}= b_f / r(p_f^\star)$;
return $p_f^\star$;

C. Unified Path Selection

The coverage result shows that the electrical layer provides local completion candidates, but it does not specify which path an online scheduler should take. That decision depends on runtime link state. Static rules that pre-assign flows by size or hop count assume path values are known in advance. Under contention, an OCS path may be unavailable while an electrical completion path offers immediate transmission. Conversely, an electrical completion path may be unattractive when its uplink is already occupied. The correct comparison is between concrete path scores, not between path classes.

Hyacinth supports two closely related selection rules. The pruned scheduler evaluates each retained candidate $p \in \tilde{P}_f$ under the local completion-time metric from Eq. (11) and selects

$$p_f^\star = \arg\min_{p \in \tilde{P}_f} \Big(\widehat{C}(f,p),\ \max\{a_f, \rho(p)\},\ |p| \Big). \tag{17}$$

The tuple compares predicted completion time first, then earliest start time, then path length. The dynamic scheduler adds a min-max reserved-load score before completion time.

$$p_f^\star = \arg\min_{p \in \tilde{P}_f} \Big(w(p), \widehat{C}(f,p), \rho(p), -r(p), |p| \Big), \tag{18}$$

where $w(p) = \max_{\ell \in p}(q_\ell + b_f / r(p))$. The two link-state variables have different roles. τ_ℓ is an availability timestamp that determines waiting in $\rho(p)$. q_ℓ is a reserved-service counter used for load balancing. It ignores release times and records how much work has been assigned to link ℓ. Thus $w(p)$ estimates post-assignment load, while $\widehat{C}(f,p)$ estimates when the flow would finish.

This distinction matters when two candidate paths are close. Suppose an electrical completion path can finish the current flow slightly earlier, but it traverses an electrical uplink that already has high reserved service. A short OCS detour may finish later for this flow while keeping the maximum post-assignment load lower. The dynamic rule can choose the OCS detour to preserve electrical completion capacity for later residual flows in overlapping collective rounds. The pruned rule is more aggressive and chooses the electrical path if its predicted completion time is lower after capping. Thus Hyacinth-dynamic favors stable shared-link pressure, whereas Hyacinth-pruned favors the best completion time for the current candidate set.

Both rules place Pure Optics and electrical completion paths under one comparison. Electrical usage is not pre-assigned. It emerges when an electrical completion path obtains a better score than the available Pure Optics alternatives under current contention. Although the scheduler is online and greedy, route choice is not an ad hoc plane-specific rule. The same candidate construction, metrics, and reservation update apply to OCS paths and electrical completion paths.

Link reservation. The selection decision depends on link state and feeds back into it. After selecting $p_f^\star$, Hyacinth updates the available time of every link on that path.

$$\tau_\ell \leftarrow \widehat{C}(f, p_f^\star), \qquad \forall \ell \text{ with } \delta_{\ell p_f^\star} = 1. \tag{19}$$

This reservation has a clear physical interpretation. The chosen path's links are occupied through the predicted completion of flow f. Subsequent flows whose candidate paths share these links observe a later $\rho(p)$ and prefer alternative paths unless waiting is genuinely the best option. The greedy sequence of per-flow decisions remains internally coherent through continuously evolving link state.

D. Scheduling Algorithms

Hyacinth supports two selection strategies that share the same reservation mechanism and compare both OCS paths and electrical completion paths, but differ in candidate generation and scoring. Algorithm 1 is the default mode and minimizes worst-case link load. Algorithm 2 is an enhanced search mode that applies per-class pruning before evaluating completion time.

The two algorithms occupy different points in the design space. The dynamic scheduler selects by min-max link load, which is effective when global load balance dominates individual flow completion time. The pruned scheduler selects by predicted completion time after per-class capping,

Algorithm 2: Hyacinth-pruned

Input : Flow $f = (s_f, d_f, b_f, a_f)$, graph G_H, link states $\{\tau_\ell\}$, per-type cap K_{type}
Output: Assigned path $p_f^\star$, updated link states

Generate candidates from the six path families;
  OCS paths include $s_f \to d_f$, $s_f \to k \to d_f$, $s_f \to k_1 \to k_2 \to d_f$;
  Electrical completion paths include $s_f \to e \to d_f$, $s_f \to k \to e \to d_f$, $s_f \to e \to k \to d_f$;
Classify each candidate into its family type;
foreach *candidate* p **do**
  $\rho(p) \leftarrow \max_{\ell \in p} \tau_\ell$;
  $r(p) \leftarrow \min_{\ell \in p} c_\ell$;
foreach *family* **do**
  Rank candidates by ($\rho(p)$ asc, $r(p)$ desc, $|p|$ asc);
  Retain top K_{type};
foreach *surviving candidate* p **do**
  $\widehat{C}(f,p) \leftarrow \max\{a_f, \rho(p)\} + b_f / r(p)$;
Select $p_f^\star$ by $(\widehat{C}(f,p), \max\{a_f, \rho(p)\}, |p|)$;
foreach *link* $\ell \in p_f^\star$ **do**
  $\tau_\ell \leftarrow \widehat{C}(f, p_f^\star)$;
return $p_f^\star$;

which is effective when path diversity matters more. We use Hyacinth-dynamic as the canonical Hyacinth configuration in the main comparison and evaluate Hyacinth-pruned separately as a mode for sparse OCS fabrics.

The per-flow cost is dominated by candidate generation. For the pruned six-family construction, the three-hop Pure Optics family can inspect at most K^2 candidates before capping, and sorting within each family costs $O(K^2 \log K)$. Evaluating the retained candidates costs $O(K_{\text{type}} \cdot |p|)$ where $|p| \le 3$. The dynamic scheduler retrieves only a capped OCS pool and capped electrical completion candidates before scoring them. For the evaluated degree-constrained topologies, both algorithms inspect a small candidate set.

V. Evaluation

This section evaluates Hyacinth with packet-level simulation across two topology scales and four traffic scenarios. We first describe the simulation setup in Section V-A, and then answer the following key questions.

Q1: Can Hyacinth reduce collective completion time across diverse traffic conditions? Section V-B compares Hyacinth with representative optical-electrical and Pure Optics baselines under fragmentation, concurrency conflict, heterogeneous model composition, and workload distribution shifts.

Q2: How does electrical tail completion affect flows of different sizes? Section V-B includes a per-flow-size breakdown that compares mice and elephant FCT.

Q3: Does bounded electrical completion help avoid deep pure-optical search? Section V-C compares Hyacinth-pruned with Optics-pruned to isolate the value of electrical completion paths under matched search structure.

Q4: Is state-dependent electrical use necessary, or can a static threshold suffice? Section V-D isolates the contribution of dynamic thresholding, and Section V-F evaluates static-threshold sensitivity through the lens of Theorem 2.

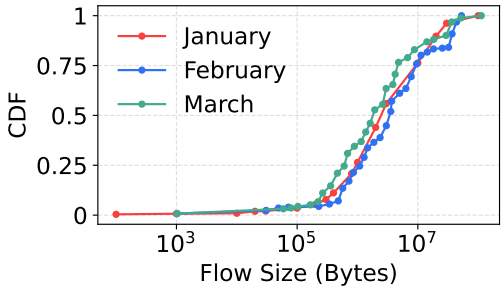

Fig. 6: Inference traffic distribution.

TABLE I: Model templates used in the evaluation.

Template	EP	PP	TP	DP	GPUs	Servers
L	64	8	1	1	512	64
M	40	8	1	1	320	40
S	32	8	1	1	256	32

Q5: Is Hyacinth lightweight enough for online scheduling? Section V-E measures the route-computation overhead of the online scheduler.

Q6: Which path families carry traffic under sparse OCS connectivity? Section V-G examines whether electrical completion replaces deep optical detours or simply acts as a separate traffic plane.

A. Simulation Setup

Simulation framework. We use htsim [4], a packet-level discrete-event simulator widely used for transport, load balancing, and topology studies. It models packet transmission, queueing, and flow completion at fine granularity, which matches our need to evaluate optical paths and electrical completion decisions. We use Bolt, a recent data-center congestion control protocol [2].

Topology configuration and scale. We evaluate two scales. The first has 40 ToRs and 2560 GPUs, with 8 hosts per rack and 8 GPUs per host, using a total core degree of 4. The second has 80 ToRs and 5120 GPUs under the same host and GPU configuration, using a total core degree of 8. In the optical-electrical configuration, the electrical layer uses $N = 1$ EPS node and replaces one OCS uplink with one electrical completion uplink while keeping the total degree unchanged. The OCS subgraph is therefore 3-regular at 2560 GPUs and 7-regular at 5120 GPUs. Pure Optics baselines dedicate the same degree budget entirely to OCS.

This degree-constrained setting matches the replacement model in Theorem 1 and Corollary 1. All ToR-ToR and ToR-EPS links operate at 100 Gbps, with 500 ns propagation delay and 200-packet queues. This equal-rate configuration isolates the effect of path-space construction and scheduling. The OCS subgraph is a random regular expander among ToRs, consistent with the K-regular model in Section III-A.

Workload scenarios. We use three model templates and evaluate Hyacinth across four workload dimensions. Table II gives the detailed settings. Each scenario varies one dimension while keeping the others at their default values.

TABLE II: Case matrix for 2560 and 5120 GPUs.

Variable	Case	2560 GPUs	5120 GPUs
Default	1	f=0.5, 3L2M1S, 100ms, Jan-2024.	f=0.5, 4L3M3S, 100ms, Jan-2024.
Frag	2–5	$f \in \{0.1, 0.3, 0.7, 0.9\}$	$f \in \{0.1, 0.3, 0.7, 0.9\}$
Composition	6–7	5L, 10S	10L, 20S
Concurrency	8–11	0, 25, 50, 75 ms	0, 25, 50, 75 ms
Workload	12–13	Feb-2024, Mar-2024	Feb-2024, Mar-2024

- **Model templates.** We use three inference templates, L, M, and S, with EP set to 64, 40, and 32. PP is set to 8, while TP and DP are set to 1. These correspond to 512, 320, and 256 GPUs, or 64, 40, and 32 servers, matching current inference configurations [57], [28], [14]. The largest per-round traffic demand is roughly $2\times$ the smallest, allowing us to test whether the dynamic threshold adapts electrical participation across communication volumes.
- **Fragmentation.** Fragmentation measures how dispersed a model deployment is across ToRs relative to the most compact feasible placement. We sweep the target fragmentation factor $f \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, where larger f spreads each model across more ToRs and increases the number of non-zero entries in the AlltoAllv demand matrix.
- **Heterogeneous model composition.** We compare mixed, Large-only, and Small-only deployments. The mixed setting uses 3 L, 2 M, and 1 S templates on 2560 GPUs, and 4 L, 3 M, and 3 S templates on 5120 GPUs. The Large-only setting uses 5 L and 10 L templates, while the Small-only setting uses 10 S and 20 S templates at the two scales, respectively.
- **Concurrency intensity.** For each inference model, the first pipeline-stage start time is sampled uniformly from $[0, s]$ ms. We use $s \in \{0, 25, 50, 75, 100\}$ ms, where smaller s creates stronger inter-model concurrency. At $s = 0$ ms, all models start simultaneously and create maximum contention; at $s = 100$ ms, inter-model staggering provides more slack.
- **Workload distribution.** We use actual production inference-traffic distributions from Meta [17] over January, February, and March 2024, shown in Fig. 6. These snapshots differ in the relative proportion of large and small inference jobs, testing whether algorithm ranking remains stable under workload-distribution shifts.

Traffic generation methodology. Given the scenario parameters above, we generate host-level AlltoAllv collective rounds with a model- and placement-aware traffic generator. Each generated flow record contains a source host, destination host, flow size in bytes, start time, and collective-group identifier. For each model instance, the generator places its hosts across ToRs under residual host-capacity constraints. The swept factor f is a target fragmentation level, while F_m below denotes the realized fragmentation:

$$F_m = \frac{t^{(m)}_{\text{used}} - t^{(m)}_{\min}}{t^{(m)}_{\max} - t^{(m)}_{\min}}, \quad \text{if } t^{(m)}_{\max} > t^{(m)}_{\min}. \tag{20}$$

Here $t^{(m)}_{\text{used}}$ is the number of ToRs used by the placement, and $t^{(m)}_{\min}$ and $t^{(m)}_{\max}$ are the minimum and maximum feasible ToR spans under residual host capacity. We set $F_m = 0$ when $t^{(m)}_{\max} = t^{(m)}_{\min}$.

After placement, each model is divided into PP groups, and each collective event corresponds to one PP-group AlltoAllv round. In the active PP group, we use top-k endpoint selection with $k = 8$: each GPU endpoint selects eight distinct destination GPU endpoints, excluding destinations on the same host; if the PP group has too few candidates, destinations are extended to other GPU endpoints of the same model. GPU-level transfers are then aggregated into host-level flows. We do not synthesize expert popularity with a Zipf model or derive bytes from token-level parameters. Instead, flow sizes are sampled from the workload CDF described above. Thus, flow-size skew comes from measured production traces, while the non-zero AlltoAllv pattern is determined by fragmented placement and PP-group-level endpoint selection.

Baselines. We compare the following algorithms under matched hardware budgets. All designs that use the electrical layer deploy $N = 1$ EPS node. For k-shortest-path (Ksp) [49], [53], [35] baselines, $k = 4$ provides a conventional reference with a static path-count budget.

- **Hybrid-ksp.** A baseline that applies k-shortest-path search with $k = 4$ over the unified optical and electrical completion graph. It uses the same degree-constrained topology as Hyacinth, but relies on a static path-count budget instead of bounded OCS and electrical completion candidate pools.
- **Optics-ksp.** A Pure Optics baseline that applies the same k-shortest-path policy to an OCS-only topology. It represents path search with a static budget in the optical graph.
- **Helios.** An optical-electrical baseline with a static threshold that partitions flows by flow size, following the pre-partitioning design of [15]. It tests whether static flow-size-based partitioning suffices for MoE inference traffic.
- **Optics-greedy.** A Pure Optics baseline that performs greedy route selection over OCS candidates only, using the same greedy logic as Hyacinth but without electrical completion paths. This isolates the contribution of electrical tail completion.
- **Optics-pruned.** A Pure Optics search variant that prunes the OCS candidate path space before route selection. It serves as the optical-only reference for the pruned search strategy.
- **Hyacinth-dynamic.** Our default production-oriented configuration. It performs greedy scheduling over the unified path graph and lets the dynamic threshold select electrical completion paths only when they improve the current path score.
- **Hyacinth-pruned.** An enhanced search mode for sparse or low-degree OCS. It applies per-class pruning to both OCS paths and electrical completion paths before completion-time-based selection.
- **Hyacinth-static-20%.** A variant of Hyacinth that uses the same OCS and electrical completion candidates but

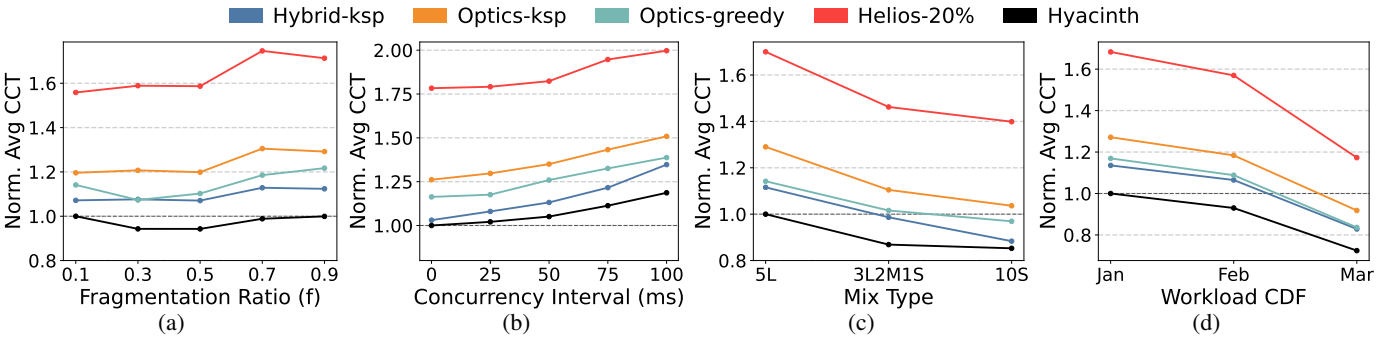

Fig. 7: Standard algorithm comparison at 2560 GPUs. (a) Fragmentation. (b) Concurrency conflict. (c) Heterogeneous model composition. (d) Workload distribution.

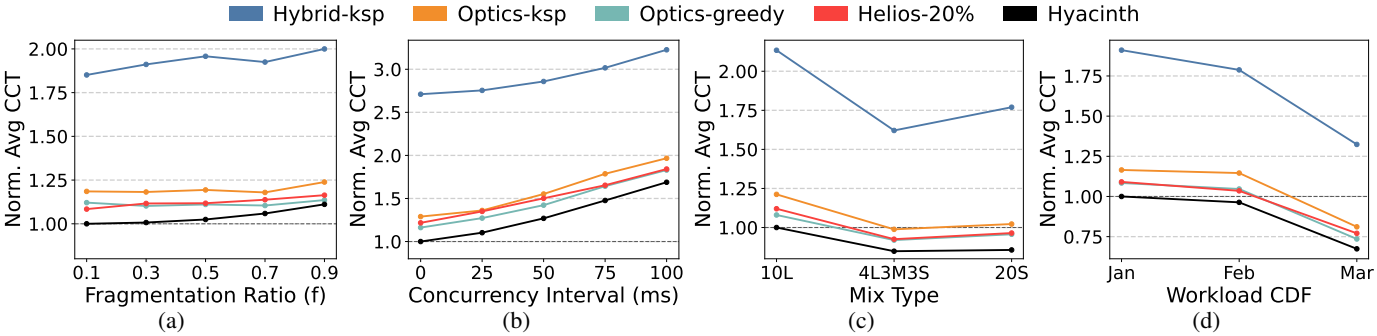

Fig. 8: Standard algorithm comparison at 5120 GPUs. (a) Fragmentation. (b) Concurrency conflict. (c) Heterogeneous model composition. (d) Workload distribution.

replaces dynamic thresholding with a static flow-size threshold. It isolates the contribution of the dynamic threshold mechanism.

B. Standard Algorithm Comparison

We compare Hyacinth against Hybrid-ksp, Optics-ksp, Helios, and Optics-greedy across the four traffic scenarios. Fig. 7 and Fig. 8 report normalized average CCT at the two topology scales. Fig. 9 gives a flow-size breakdown. All CCT values are normalized to Hyacinth's scenario mean.

Fragmentation. Fig. 7a and Fig. 8a show normalized CCT as model deployments become more dispersed across ToRs. Hyacinth achieves the lowest CCT at both scales. On 2560 GPUs, Hybrid-ksp, Optics-greedy, Optics-ksp, and Helios incur 11.1%, 15.0%, 21.5%, and 40.7% higher CCT than Hyacinth.

The gap grows as fragmentation increases from $f = 0.1$ to $f = 0.9$. Fragmented placements create more source-destination pairs without direct OCS links, which pushes Pure Optics toward congested multi-hop paths. Hyacinth instead routes residual flows through short electrical completion paths when they reduce completion time. On 5120 GPUs, Hybrid-ksp is 46.6% above Hyacinth because its static $k = 4$ path budget misses useful alternatives in the 7-regular OCS graph. Hyacinth remains stable by keeping separate bounded pools for OCS paths and electrical completion paths.

Concurrency conflict. Fig. 7b and Fig. 8b show normalized CCT as s decreases from 100 ms to 0 ms. Stronger concurrency increases CCT for all algorithms, but Hyacinth degrades the least. On 2560 GPUs, Hyacinth's CCT rises by 18.7% from the most relaxed to the most synchronized setting. Optics-ksp rises by 42.5%, and Helios rises by 89.3%.

Across the same scale, Hyacinth outperforms Hybrid-ksp by 6.6%, Optics-greedy by 17.8%, Optics-ksp by 27.8%, and Helios by 75.8%. The unified candidate set gives the greedy scheduler more ways to avoid overlapping bottlenecks, while the dynamic threshold uses electrical completion paths when OCS links are occupied. On 5120 GPUs, Hybrid-ksp incurs a 138.6% penalty because its static $k = 4$ path budget cannot capture enough path diversity under dense conflict. Pure Optics benefits from the larger OCS degree, but still lacks electrical completion capacity for absorbing small-flow queueing pressure.

Heterogeneous model composition. Fig. 7c and Fig. 8c compare mixed, homogeneous Large-only, and homogeneous Small-only compositions. Hyacinth has the lowest CCT in all three cases at both scales. On 2560 GPUs, Hyacinth leads Hybrid-ksp by 7.7%, Optics-greedy by 14.2%, Optics-ksp by 25.2%, and Helios by 67.2%.

The largest gap with Helios appears in the mixed setting. Medium flows from the M template often fall near the static 20th percentile boundary, where pre-partitioning is brittle. Hyacinth routes each flow based on actual path availability instead of a hard class label. On 5120 GPUs, Hyacinth leads Hybrid-ksp by 111.1%, Optics-ksp by 20.6%, Helios by 12.6%, and Optics-greedy by 10.2%. The consistent gap

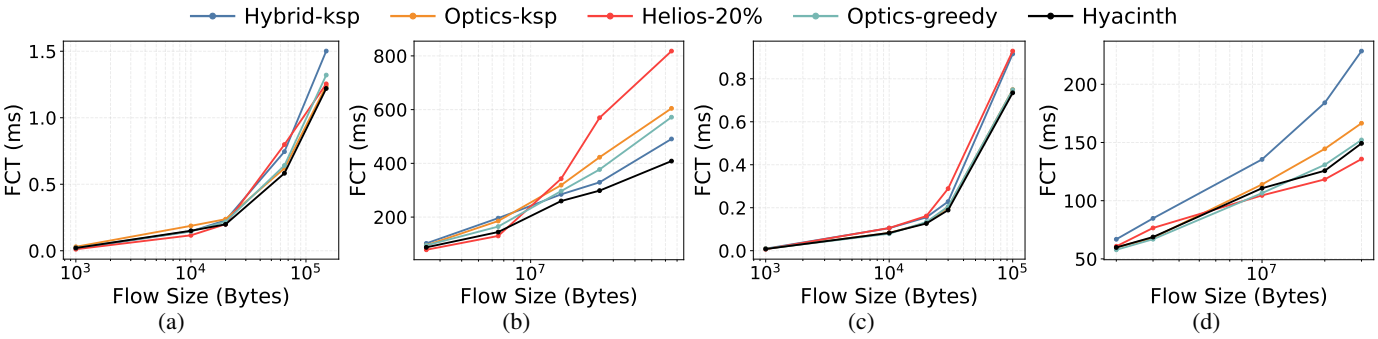

Fig. 9: CDF analysis by flow size. (a) Mice flows at 2560 GPUs. (b) Elephant flows at 2560 GPUs. (c) Mice flows at 5120 GPUs. (d) Elephant flows at 5120 GPUs.

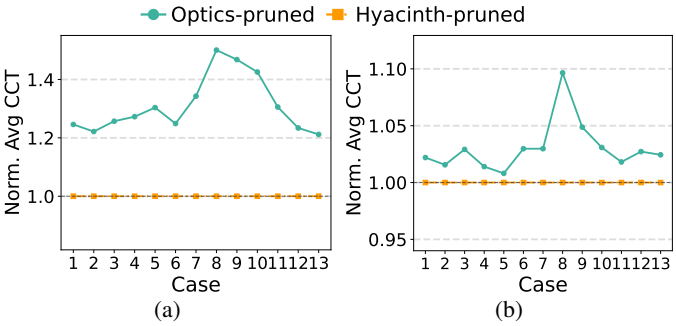

Fig. 10: Search variants across two topology scales. (a) Norm. Avg. CCT at 2560 GPUs. (b) Norm. Avg. CCT at 5120 GPUs.

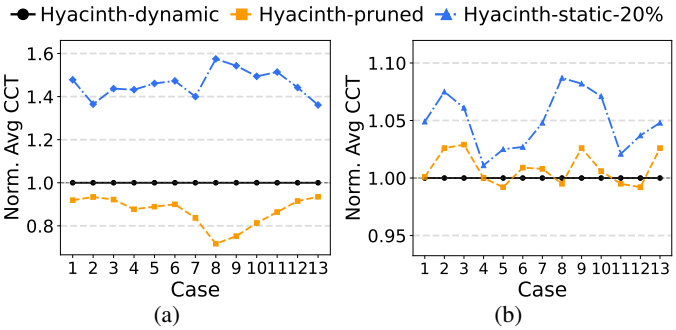

Fig. 11: Three Hyacinth variants across two topology scales. (a) Norm. Avg. CCT at 2560 GPUs. (b) Norm. Avg. CCT at 5120 GPUs.

across mixture ratios shows that Hyacinth's benefit does not depend on a single flow-size distribution.

Workload distribution shift. Fig. 7d and Fig. 8d compare monthly workload CDFs from production data. Hyacinth leads at both scales, and the ranking remains stable across snapshots. At 2560 GPUs, Hybrid-ksp trails by 14% to 15%, Optics-greedy by 16% to 17%, Optics-ksp by about 27%, and Helios by 63% to 69%.

At 5120 GPUs, Hyacinth again leads, with Helios and Optics-greedy close behind at roughly 1.09. The dynamic threshold responds to per-round queue state rather than aggregate traffic statistics, so it adapts as the workload CDF shifts. A pre-partitioning scheme such as Helios would need to retune its threshold when the proportion of large jobs changes. This stability is important for production inference workloads that evolve over time.

Per-flow-size breakdown. Fig. 9 breaks down flow completion time by flow size, comparing Hyacinth against Optics-ksp and Optics-greedy. On 2560 GPUs, Hyacinth reduces FCT for 65 KB mice flows by 6.0% over Optics-ksp and 9.0% over Optics-greedy. For 65 MB elephant flows, the reductions are 32.6% and 28.7%.

On 5120 GPUs, Hyacinth reduces FCT for 30 KB mice flows by 6.4% over Optics-ksp and 7.8% over Optics-greedy. For 20 MB elephant flows, the reductions are 13.1% and 3.8%. The elephant-flow gain is larger at 2560 GPUs because the 3-regular OCS graph provides fewer alternatives, making electrical completion more valuable. At 5120 GPUs, the 7-regular graph offers more optical paths, but Hyacinth still outperforms both Pure Optics baselines. The same mechanism holds across scales. Electrical completion reduces queueing when optical detours are costly, while OCS remains available for flows that benefit from optical transmission.

Hyacinth achieves the lowest or tied-lowest normalized average CCT across all four scenarios and both topology scales. The advantage comes from the joint effect of the dynamic threshold and the unified candidate set. The threshold avoids static flow-label mistakes by adapting to runtime link state. The bounded candidate set preserves diversity across OCS paths and electrical completion paths without relying on a static path-count budget. Together, these mechanisms realize the tail-completion principle of Section II-B. The electrical layer participates only when it can replace deeper OCS search.

C. Search Algorithm Comparison

We next evaluate Hyacinth-pruned as an enhanced search mode. We compare it with Optics-pruned. The two variants apply the same pruning logic before route selection, but one searches the unified optical and electrical completion candidate space while the other searches only OCS paths. This isolates the value of electrical completion paths under matched search structure.

Performance across scales. Fig. 10a and Fig. 10b compare Hyacinth-pruned with Optics-pruned. On 2560 GPUs, Hyacinth-pruned outperforms Optics-pruned by 31.1% overall, with a per-scenario mean of 1.000 versus 1.311. The gains

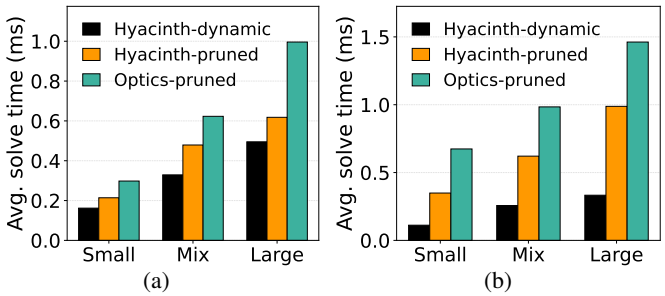

Fig. 12: Average scheduler solve time. (a) 2560 GPUs. (b) 5120 GPUs.

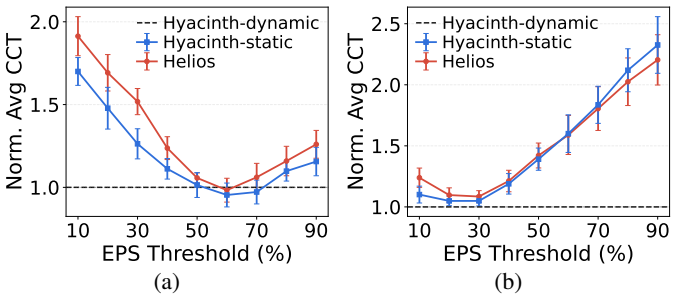

Fig. 13: Static electrical threshold sensitivity on the Default case. (a) 2560 GPUs. (b) 5120 GPUs.

are 20.6% in Fragmentation, 29.8% in Conflict, 22.8% in Heterogeneous model composition, and 18.2% in Workload.

The largest gain appears under Conflict because electrical completion paths provide a congestion escape route that Pure Optics search cannot match. On 5120 GPUs, the overall gap narrows to 3.0%, with means of 1.000 and 1.030 and a per-scenario maximum of 9.6%. This scale dependence matches Corollary 1. Electrical completion can replace deep optical detours with $O(K)$ local completion candidates, so it is most valuable when K is smaller and the optical graph is sparser.

Adding electrical completion paths to pruned search reduces CCT in all tested scenarios. The benefit is strongest under conflict, where path diversity directly mitigates queue buildup. The pruned mode complements the dynamic mode when the optical graph is sparse and higher route quality is worth the extra search.

D. Hyacinth Variant Comparison

We compare three internal strategies. The dynamic variant refers to Hyacinth-dynamic, the default production-oriented scheduler with a dynamic threshold. The pruned variant refers to Hyacinth-pruned, the enhanced search mode over the unified candidate space. The static-20% variant refers to Hyacinth-static-20%, which keeps the same candidates but replaces dynamic thresholding with a static 20% split. These comparisons isolate the value of the dynamic threshold and when extra search is beneficial.

Performance across scales. Fig. 11a and Fig. 11b report overall means. On 2560 GPUs, the dynamic variant achieves 1.000, the pruned variant improves over it by 13.3% at 0.867, and static-20% trails it by 46.0% at 1.460. On 5120 GPUs, the dynamic variant achieves 1.000 and outperforms the pruned variant by 0.8% at 1.008 and static-20% by 4.9% at 1.049.

The crossover reflects the available optical path space. At 2560 GPUs, the 3-regular OCS graph is sparse, so the pruned mode can find routes that greedy scheduling misses. At 5120 GPUs, the 7-regular graph provides enough diversity for greedy selection to work well. The dynamic threshold is the primary driver of Hyacinth's performance. Even without search-based selection, the dynamic variant substantially outperforms static-20% at both scales.

Scenario-level breakdown. On 2560 GPUs, the pruned variant improves over the dynamic variant in all four scenarios. It achieves 0.908 in Fragmentation for a 9.2% gain, 0.786 in Conflict for a 21.4% gain, 0.869 in Heterogeneous model composition for a 13.1% gain, and 0.925 in Workload for a 7.5% gain. The static-20% variant consistently underperforms the dynamic variant, with penalties of 43.5%, 53.1%, 43.7%, and 40.1%.

On 5120 GPUs, the dynamic variant is optimal or tied for optimal in all four scenarios. The pruned variant trails by 1.0% in Fragmentation, 0.6% in Conflict, 0.8% in Heterogeneous model composition, and 0.9% in Workload. The static-20% variant trails by 4.4%, 6.5%, 3.8%, and 4.2%. The largest static-20% penalty appears under Conflict because concurrency-driven queue pressure is exactly what the dynamic threshold adapts to avoid. When collective rounds overlap, the dynamic variant routes boundary flows according to current OCS link availability, while static-20% assigns them rigidly by the static split.

These results clarify the role of the two modes. Hyacinth-dynamic is the canonical configuration because it is stable across scales and captures most of the benefit through dynamic thresholding. Hyacinth-pruned is useful for sparse OCS fabrics, where extra search can find better routes than greedy selection. Thus the modes are complementary rather than competing definitions of Hyacinth.

E. Solving Time

We measure the average route computation time per scheduling call in the heterogeneous model composition scenario. This experiment uses the same ten seeds as the CCT experiments and excludes packet level simulation time. We compare Hyacinth-dynamic, Hyacinth-pruned, and Optics-pruned, since their online cost depends on candidate enumeration.

Fig. 12 shows that Hyacinth-dynamic adds below one millisecond of scheduling overhead. Its maximum solve time is 0.495 ms at 2560 GPUs and 0.333 ms at 5120 GPUs. Averaged over the three model composition cases, the solve time is 0.329 ms and 0.234 ms. This follows from the dynamic mode's bounded candidate pools and simple link state scoring.

Hyacinth-pruned costs more because it ranks the pruned candidate set by completion time, but it remains below 0.62 ms at 2560 GPUs and below 0.99 ms at 5120 GPUs. It is also faster than Optics-pruned in every case. Averaged over the three cases, Hyacinth-pruned reduces solve time from 0.639 to 0.437 ms at 2560 GPUs and from 1.040 to 0.653 ms at 5120 GPUs. This is consistent with Corollary 1 and Lemma 1. Electrical completion candidates contract deep optical expansion

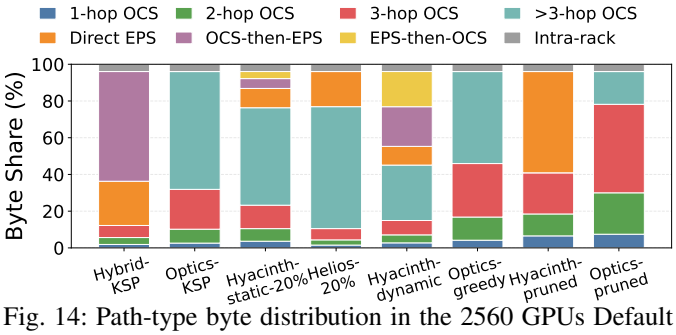

Fig. 14: Path-type byte distribution in the 2560 GPUs Default case.

into a bounded local comparison while preserving the same completion time metric.

F. Threshold Sensitivity

We test whether a static electrical threshold can replace the dynamic threshold. We sweep the threshold from 10% to 90% on the Default case at both topology scales. Helios uses the threshold to partition flows by size. Hyacinth-static uses the same static threshold inside Hyacinth's unified candidate space. All values are normalized to Hyacinth-dynamic at the same scale.

Fig. 13 shows that the best static threshold changes with topology scale. On 2560 GPUs, where one electrical uplink is paired with a 3-regular OCS subgraph, the best sampled point is between 50% and 60%. Hyacinth-static reaches 0.954 at 60%, while Helios reaches 0.983. This setting has fewer optical alternatives, so using the electrical layer for more residual flows can help. However, thresholds below 40% underuse electrical completion and thresholds above 70% begin to concentrate traffic on the shared electrical layer.

The same threshold does not transfer to the 5120-GPU topology. With one electrical uplink and a 7-regular OCS subgraph, the best region shifts to between 20% and 30%. Hyacinth-static is 1.049 at both 20% and 30%, while Helios reaches 1.085 at 30%. Larger thresholds quickly degrade CCT because the denser OCS graph already provides more optical alternatives, and excessive electrical use creates shared uplink contention. This is consistent with Theorem 2. A single static threshold is brittle because the effective threshold moves with link availability. Hyacinth-dynamic avoids retuning for each topology by selecting electrical completion paths from current link state rather than from a preset split.

G. Path-Type Distribution

We finally examine which path families carry traffic in the sparse 2560 GPUs Default case. Because contention is driven by traffic volume rather than by the number of flows alone, Fig. 14 reports byte shares. The figure includes intra-rack traffic for completeness; these bytes do not traverse the core fabric.

Fig. 14 shows that Pure Optics baselines rely heavily on deep optical detours in the sparse OCS fabric. Optics-ksp sends 64.2% of bytes over optical paths longer than three hops, and Optics-greedy sends 50.1%. Optics-pruned reduces this tail but still leaves 17.8% of bytes on longer-than-three-hop optical paths. These long paths are exactly the residual cases where deeper optical search expands the candidate space and increases exposure to shared optical bottlenecks.

Hyacinth replaces this deep optical tail with bounded electrical completion paths rather than assigning all traffic to the electrical plane. Hyacinth-pruned eliminates longer-than-three-hop optical paths in this case and carries 55.1% of bytes on direct electrical completion paths. Hyacinth-dynamic uses a more balanced mix: 10.2% of bytes use direct electrical completion, 21.6% use OCS-then-electrical paths, and 19.2% use electrical-then-OCS paths, while 45.1% remain on pure OCS paths. In contrast, Hybrid-ksp sends 83.8% of bytes through electrical completion paths, and Helios-20% still leaves 66.4% of bytes on longer-than-three-hop OCS paths. This supports the design goal of using electrical completion as a state-dependent residual-flow mechanism, not as a preset traffic plane.

The same routing logs show the corresponding reduction in optical path length. Compared with Optics-pruned, Hyacinth-pruned reduces the average OCS hop count from 2.81 to 1.08 on core traffic. Compared with Optics-greedy, Hyacinth-dynamic reduces it from 3.39 to 2.45. Thus the electrical completion layer is not merely adding another path class; it directly replaces deep optical detours for residual traffic.

VI. Related Work

Coflow and Communication Scheduling. The coflow abstraction [12] captures application-level communication semantics by grouping flows with a common performance goal. Orchestra [10], Varys [11], and Aalo [13] design increasingly capable coflow schedulers, while Rapier [59] jointly optimizes routing and scheduling, and Sincronia [1] improves coflow completion through network design. For OCS-based networks, Sunflow [19] schedules coflows with not-all-stop reconfiguration, and Reco [51] provides regularization-based approximation algorithms. Recent approximation algorithms also study hybrid optical-electrical switches with worst-case guarantees [55].

Optical Circuit Switching in Data Centers. RotorNet [34] demonstrates scalable optical networking using free-space optics, while Jupiter Evolving [40] and Lightwave Fabrics [31] showcase at-scale OCS deployment and its extension to ML systems. TPU v4 [23] deploys optically reconfigurable interconnects for ML supercomputers. Realizing RotorNet [36] advances practical microsecond-scale optical networking. Sirius [3] targets nanosecond-scale OCS with flat topologies, and OSA [7] enables flexible switching through dynamic wavelength assignment.

Hybrid Optical-Electrical Architectures. Helios [15] and c-Through [54] pioneer hybrid optical-electrical topologies by routing large flows over optical paths and small flows over electrical paths. REACToR [29] demonstrates microsecond-scale hybrid ToR switching by synchronizing end-host transmissions with circuit assignments. Solstice [30] schedules hybrid circuit/packet networks using stable-marriage-inspired

matching, and MixNet [27] proposes a runtime-reconfigurable optical-electrical fabric with regional OCS domains for MoE training. These systems mainly decide when traffic should use optical circuits, when it should use packet switching, or how topology should be reconfigured.

Communication Optimization for Distributed ML Traffic. TACCL [47] synthesizes collective communication algorithms for heterogeneous topologies, while ByteScheduler [39] and SYNDICATE [33] optimize tensor transmission ordering and execution planning. Crux [5] mitigates contention among concurrent training jobs with GPU-intensity-aware scheduling. CASSINI [43] and MLTCP [42] address network-aware job scheduling and congestion control, while RailS [56] exploits rail-aware multipath scheduling for sparse all-to-all communication in MoE training. For MoE architectures, GShard [24], Switch Transformer [16], and MegaScale [22] scale sparse models and large LLM training, while Tutel [21], FasterMoE [18], and DeepSpeed-MoE [45] optimize MoE communication through adaptive parallelism, expert placement, and hierarchical all-to-all.

VII. Conclusion

This paper presented Hyacinth, an online residual-flow completion scheduler for MoE inference over optical interconnects. We showed that the hard part of pure-optical scheduling is not serving the well-matched flows, but completing residual flows without expanding deep optical search. Hyacinth addresses this bottleneck by placing OCS paths and electrical completion paths in one unified graph, then scheduling flows by predicted completion time under current link state. In this view, a small electrical layer is a limited completion resource and a source of bounded local candidates, rather than a separate traffic plane. Packet-level evaluation across two topology scales and four traffic scenarios shows that this design consistently reduces collective completion time over Pure Optics and optical-electrical baselines with static thresholds. More broadly, Hyacinth suggests that future optical inference fabrics should co-design optical path diversity with limited electrical completion capacity.

References

[1] Saksham Agarwal et al. Sincronia: Near-optimal network design for coflows. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 16–29, 2018.

[2] Serhat Arslan et al. Bolt:{Sub-RTT} congestion control for {Ultra-Low} latency. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 219–236, 2023.

[3] Hitesh Ballani et al. Sirius: A flat datacenter network with nanosecond optical switching. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 782–797, 2020.

[4] Broadcom. htsim. https://github.com/Broadcom/csg-htsim, 2023.

[5] Jiamin Cao et al. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 1–15, 2024.

[6] Aili Chen et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

[7] Kai Chen et al. Osa: An optical switching architecture for data center networks with unprecedented flexibility. *IEEE/ACM Transactions on networking*, 22(2):498–511, 2013.

[8] Li Chen et al. Enabling {Wide-Spread} communications on optical fabric with {MegaSwitch}. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 577–593, 2017.

[9] Wei Chen, Ye Tian, and Xinming Zhang. Acceltor: Accelerating tcp for circuit/packet hybrid data centers with packet scheduling. *IEEE Transactions on Networking*, 2025.

[10] Mosharaf Chowdhury et al. Managing data transfers in computer clusters with orchestra. *ACM SIGCOMM computer communication review*, 41(4):98–109, 2011.

[11] Mosharaf Chowdhury et al. Efficient coflow scheduling with varys. In *Proceedings of the 2014 ACM conference on SIGCOMM*, pages 443–454, 2014.

[12] Mosharaf Chowdhury and Ion Stoica. Coflow: A networking abstraction for cluster applications. In *Proceedings of the 11th ACM Workshop on Hot Topics in Networks*, pages 31–36, 2012.

[13] Mosharaf Chowdhury and Ion Stoica. Efficient coflow scheduling without prior knowledge. *ACM SIGCOMM Computer Communication Review*, 45(4):393–406, 2015.

[14] Nan Du et al. Glam: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pages 5547–5569, 2022.

[15] Nathan Farrington et al. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 339–350, 2010.

[16] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[17] Adithya Gangidi et al. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, 2024.

[18] Jiaao He et al. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 120–134, 2022.

[19] Xin Sunny Huang et al. Sunflow: Efficient optical circuit scheduling for coflows. In *Proceedings of the 12th International on Conference on emerging Networking EXperiments and Technologies*, pages 297–311, 2016.

[20] Yanping Huang et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

[21] Changho Hwang et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems*, 5:269–287, 2023.

[22] Ziheng Jiang et al. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–760, 2024.

[23] Norm Jouppi et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*, pages 1–14, 2023.

[24] Dmitry Lepikhin et al. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[25] Jialong Li et al. Uniform-cost multi-path routing for reconfigurable data center networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 433–448, 2024.

[26] Shen Li et al. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704*, 2020.

[27] Xudong Liao et al. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 554–574, 2025.

[28] Aixin Liu et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[29] He Liu et al. Circuit switching under the radar with {REACToR}. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, pages 1–15, 2014.

[30] He Liu et al. Scheduling techniques for hybrid circuit/packet networks. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*, pages 1–13, 2015.

[31] Hong Liu et al. Lightwave fabrics: at-scale optical circuit switching for datacenter and machine learning systems. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 499–515, 2023.

[32] Yuhan Liu et al. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.

[33] Kshiteej Mahajan et al. Better together: Jointly optimizing {ML} collective scheduling and execution planning using {SYNDICATE}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 809–824, 2023.
[34] William M Mellette et al. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 267–280, 2017.
[35] William M Mellette et al. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1–18, 2020.
[36] William M Mellette et al. Realizing rotornet: Toward practical microsecond scale optical networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 392–414, 2024.
[37] Deepak Narayanan et al. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM symposium on operating systems principles*, pages 1–15, 2019.
[38] Pratyush Patel et al. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.
[39] Yanghua Peng et al. A generic communication scheduler for distributed dnn training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 16–29, 2019.
[40] Leon Poutievski et al. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 66–85, 2022.
[41] Ruoyu Qin et al. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage*, 2024.
[42] Sudarsanan Rajasekaran et al. Mltcp: A distributed technique to approximate centralized flow scheduling for machine learning. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*, pages 167–176, 2024.
[43] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. {CASSINI}:{Network-Aware} job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1403–1420, 2024.
[44] Samyam Rajbhandari et al. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
[45] Samyam Rajbhandari et al. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*, pages 18332–18346. PMLR, 2022.
[46] R Ryf et al. 1296-port mems transparent optical crossconnect with 2.07 petabit/s switch capacity. In *OFC 2001. Optical Fiber Communication Conference and Exhibit. Technical Digest Postconference Edition (IEEE Cat. 01CH37171)*, volume 4, pages PD28–PD28. IEEE, 2001.
[47] Aashaka Shah et al. {TACCL}: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, 2023.
[48] Mohammad Shoeybi et al. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
[49] Ankit Singla et al. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 225–238, 2012.
[50] Foteini Strati et al. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving. In *Proceedings of the 41st International Conference on Machine Learning*, pages 46745–46771, 2024.
[51] Haisheng Tan et al. Regularization-based coflow scheduling in optical circuit switches. *IEEE/ACM Transactions on Networking*, 29(3):1280–1293, 2021.
[52] Kimi Team et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
[53] Asaf Valadarsky et al. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International on Conference on emerging Networking EXperiments and Technologies*, pages 205–219, 2016.
[54] Guohui Wang et al. c-through: Part-time optics in data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 327–338, 2010.
[55] Xin Wang, Hong Shen, and Hui Tian. Scheduling coflows in hybrid optical-circuit and electrical-packet switches with performance guarantee. *IEEE/ACM Transactions on Networking*, 32(3):2299–2314, 2024.
[56] Heng Xu et al. Rails: Load balancing for all-to-all communication in distributed mixture-of-experts training. *IEEE Transactions on Networking*, 34:4431–4448, 2026.
[57] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
[58] Aohan Zeng et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
[59] Yangming Zhao et al. Rapier: Integrating routing and scheduling for coflow-aware data center networks. In *2015 IEEE Conference on Computer Communications (INFOCOM)*, pages 424–432. IEEE, 2015.
[60] Yinmin Zhong et al. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.